

Software Architecture- 04

KALINGA GUNAWARDHANA

DEPT OF COMPUTING & INFORMATION SYSTEMS

SABARAGAMUWA UNIVERSITY OF SRI LANKA

A solid green horizontal bar at the bottom of the slide.

Abstract Data Types and OO Design

Object-oriented design architectural styles are focused on the data.

Steps when modelling a system

- Begin looking at the different kinds of data handled by the system to see how the system can be broken down into abstract data types.
- An abstract data type can be represented as a class that you define to organize data attributes in a meaningful way
- Encapsulation restricts access to the data and dictates what can be done with it.

Object-oriented refers to a system composed of objects where each object is an instance of a class. In other words, the type of each object is its class.

Objects may interact with each other through the use of their methods. The object-oriented paradigm allows for inheritance among abstract data types. This means that one abstract type can be declared an extension of another type.

Note

Some problems are well suited to an object-oriented architectural style. However, not all situations will have easily identifiable classes. In some situations, another design choice may be better suited for the problem at hand.

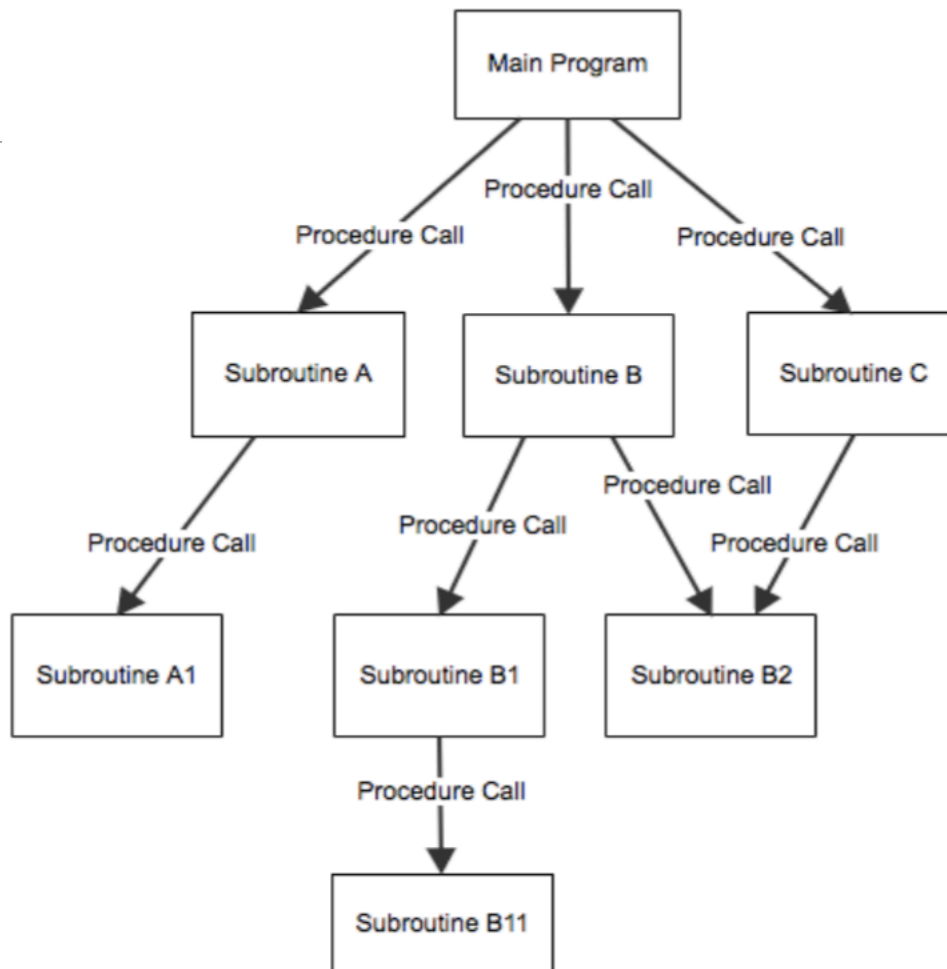
Main program and Subroutine

Main program and subroutine architectural styles are focused on functions.

They develop from a procedural programming paradigm. (ex C)

the terms routine, procedure, and function are used interchangeably with the term subroutine.

In a main program and subroutine architectural style, a system is modelled by breaking up the overall functionality of the system into a main program and subroutines



-
- In this paradigm, data is stored as variables
 - Abstract data types are supported in procedural programming; however, inheritance is not explicitly supported.
 - The main focus of this paradigm is therefore on the behaviour of functions and how data moves through those functions.
 - The main program and subroutine architectural style is best suited for computation-focused systems.

-
- Each subroutine may have its own local variables.
 - A subroutine has access to all data within its scope.
 - To access data outside its scope, data may be passed into the subroutines as parameters, by value or by reference, and data may be passed out of a subroutine as a return value.
 - A principle of this paradigm is “one entry, one exit per subroutine,” which makes the control flow of the subroutine easier to follow.

Advantages

When functions are well defined and do not produce side effects, they are considered like “black boxes.”

Given an input, they will always have the same output. Further, library functions are easily integrated into programs.

Disadvantages

Subroutines may mutate data in unexpected ways

- A subroutine may be affected by data changes made by another subroutine during execution.
- This issue is especially true for global data shared across subroutines.
- Changes to data can be unpredictable and result in a function receiving an input that it was not expecting, or even an input that it is not capable of handling correctly, which can result in run-time errors.

Repository-Based Systems

Modern software systems rarely operate in isolated environments. As a software developer, any software architecture you create needs to be capable of sharing information between separate components.

A number of issues need to be addressed to do this:

- The state of a component is only temporary while the component is running. This means that any objects created and any amount of data processing will be lost when the component stops running.
- Components may not need all the data that is stored in the files. There needs to be a way to communicate the state of the data between multiple components.

To solve both of the issues, a **data-centric software architecture** can be used.

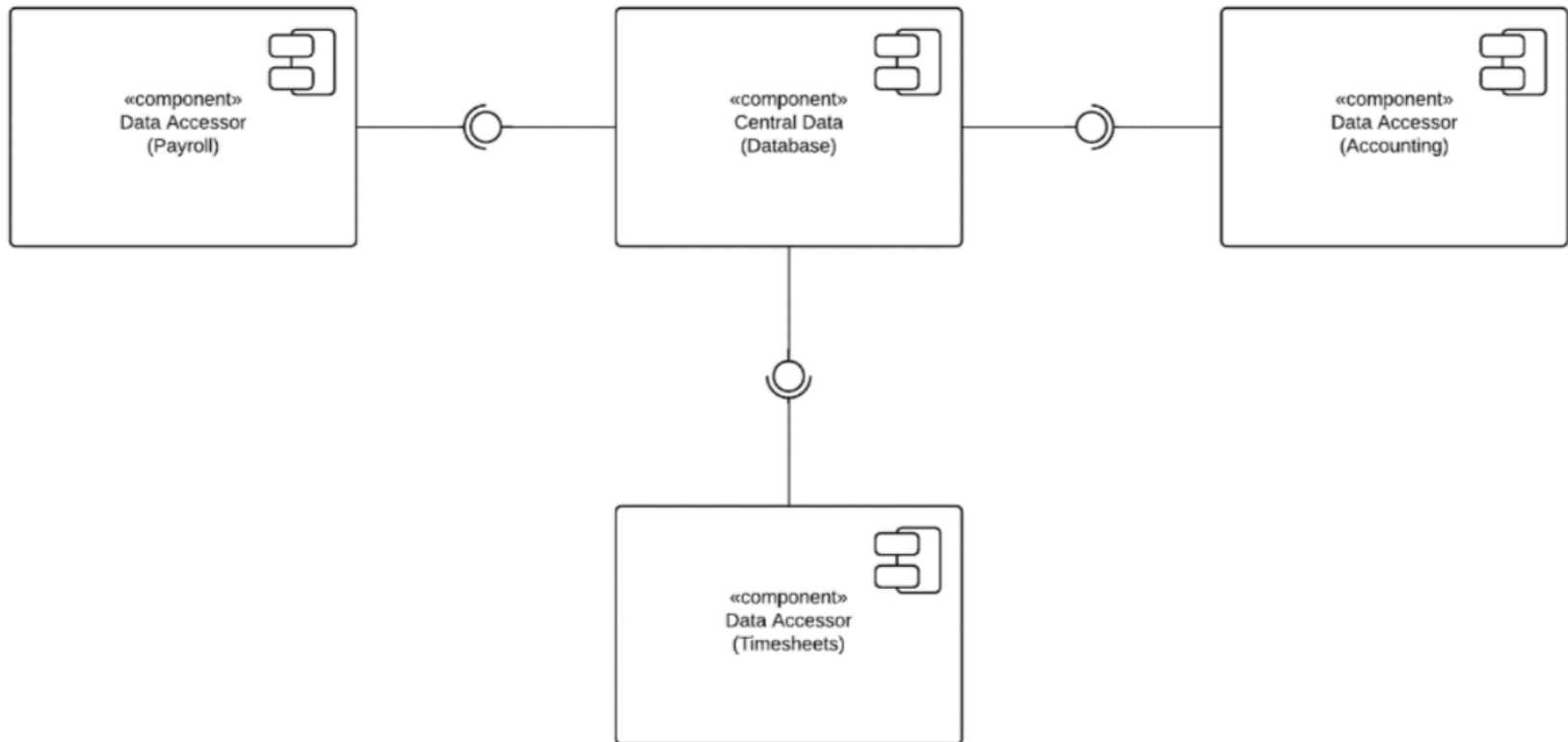
This architecture allows data to be stored and shared between multiple components, thus increasing the **maintainability**, **reusability**, and **scalability** of the system.

This architecture can be achieved by integrating a method of shared data storage, such as a database, into the overall system design.

A solid green horizontal bar at the bottom of the slide.

At the core of a data-centric architecture are two types of components:

- **Central data** is the component used to store and serve data across all components that connect to it.
- **Data accessors** are the components that connect to the central data component. The data accessors make queries and transactions against the information stored in the database.



Databases

Central data is often stored in a **database**. Databases ensure several data qualities, which are important to data-centric architectures. These are:

- **Data integrity.** A database ensures that data is accurate and consistent over its lifespan.
- **Data persistence.** A database ensures that data will live on after a process has been terminated—databases can save data from any number of components.

In data-centric architectural design, the central data is passive. Central data is primarily concerned with storing and serving information, not with heavy data processing or large amounts of business logic.

Data accessors are any component that connects to a database, which is characterized by its abilities to:

- Share a set of data, while being able to operate independently.
- Communicate with the database through database queries and transactions. Data accessors do not need to interact directly with each other and therefore do not need to know about each other.

- Query the database to obtain shared system information. This is used to get data in order to perform computations.

- Save the new state of the system back into the database using transactions. Data is stored back into the database once the data accessor has finished its processing.

A data accessor contains all the business rules required to perform its functions. This means that this software architecture enables you to separate concerns into different, specialized data accessors.

Advantages

Data-centric architecture presents many advantages over a basic object-oriented system, because of the integration of a centralized database. These advantages include:

- Increased support of data integrity, data backup, and data restoration.
- Reduced overhead for data transfer between data accessor, as data accessors do not need to be concerned with talking to one another—the database communicates for them,
- A system that can be easily scaled up, as data accessors are functionally independent, so additional features can be added with having to worry about affecting others.
- Central data components usually “live” on a separate server machine with sufficient disk storage dedicated to the database, which allow for easier management of information.

Disadvantages

- The system is heavily reliant on the central data component, so if it becomes unavailable or if the data corrupts, then the entire system is affected. Safeguards such as data redundancies that replicate data on separate hard disks can be costly.
- Data accessors are dependent on what gets stored in the database. New data accessors need to build around the existing data schema. Anything that isn't stored needs to be computed, or if there is no matching column or table for a specific data need, then the database cannot be used.
- It is difficult to change the existing data schema, particularly if a large amount of data is already stored. Further, data schema changes will affect data accessors.

Layered Systems
